



LAB 2

MICROPROCESSORS – MICROCONTROLLERS

LECTURER: NGUYỄN HÒA HƯNG
PHAN VĂN SỸ

GROUP: CC03

STUDENT

NAME: HUỲNH TRÍ ĐỨC

ID: 2452269

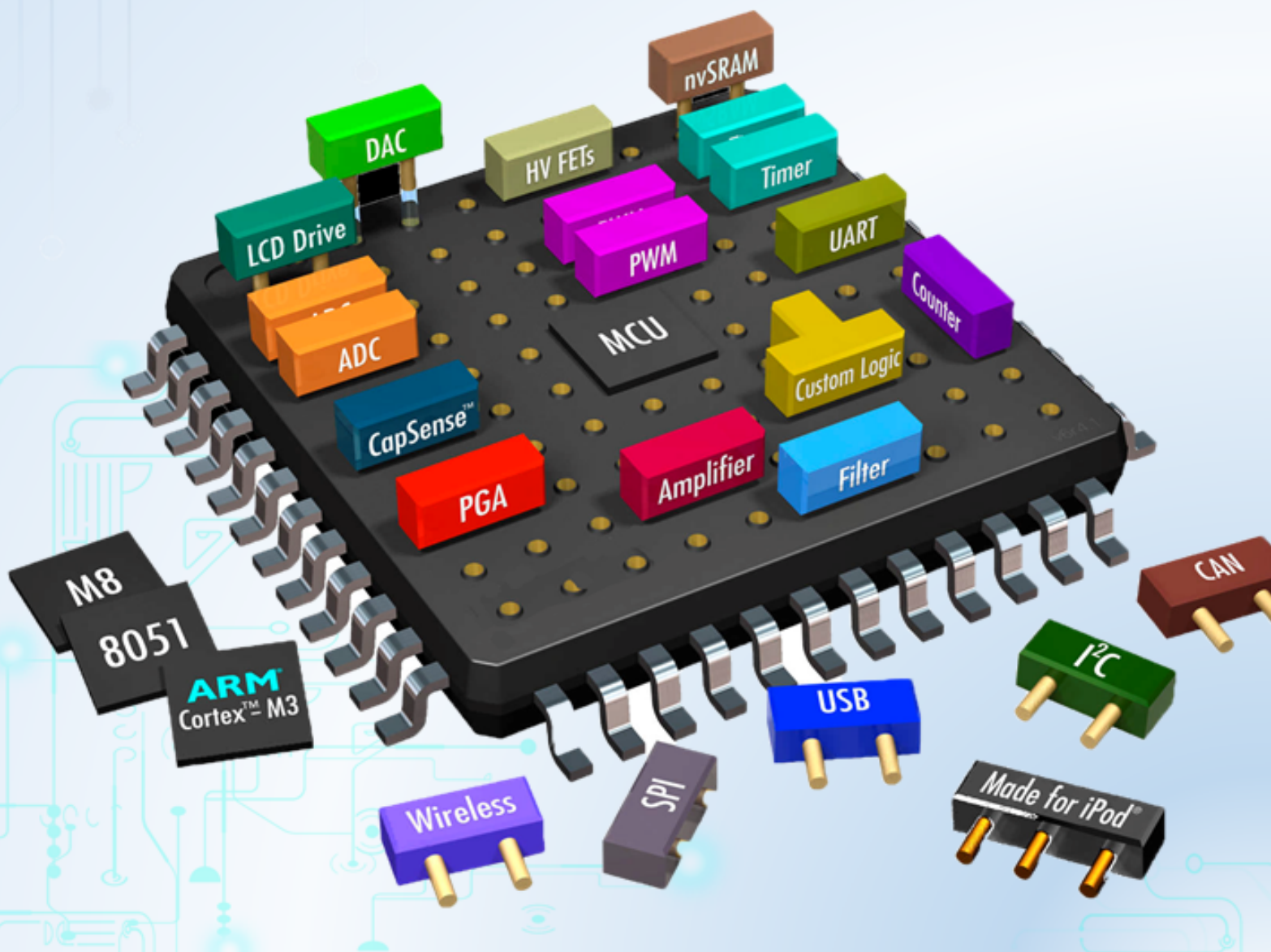
September 18, 2026

Contents

Chapter 1. Timer Interrupt and LED Scanning	3
1 Exercise and Report	4
1.1 Exercise 1	4
1.2 Exercise 2	8
1.3 Exercise 3	12
1.4 Exercise 4	17
1.5 Exercise 5	21
1.6 Exercise 6	26
1.7 Exercise 7	32
1.8 Exercise 8	37
1.9 Exercise 9	42
1.10 Exercise 10	46

CHAPTER 1

Timer Interrupt and LED Scanning



1 Exercise and Report

1.1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the signal pins. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following:

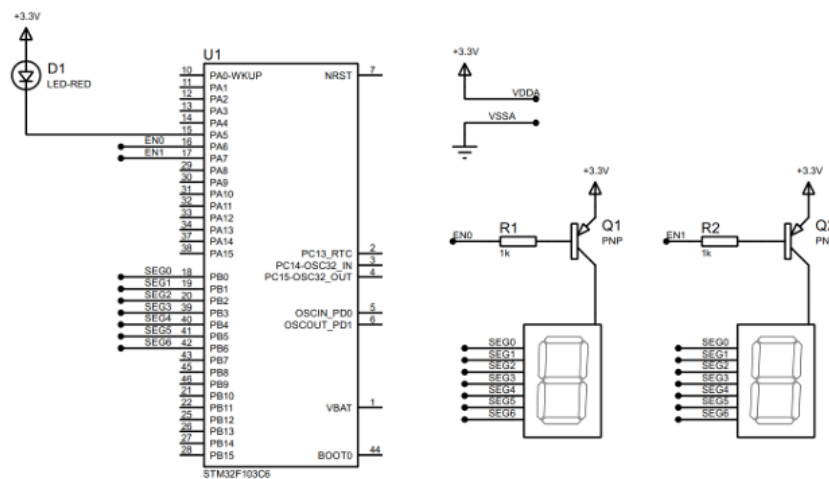


Figure 1.1: Simulation schematic in Proteus

Components used in the schematic are listed below:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED
- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** from Lab 1 in this exercise. Implement the source code in the interrupt callback function to display the number “1” on the first seven-segment display and the number “2” on the second one. The switching time between the two displays is half a second.

Report 1: Capture your schematic from Proteus and present it in the report.

Report 2: Present your source code in the HAL_TIM_PeriodElapsedCallback function.

Short question: What is the frequency of the scanning process?

Each 7-segment LED is displayed for 0.5 seconds. Since there are 2 LEDs, a complete scanning cycle takes 1 second. Therefore, the scanning frequency is 1 Hz.

Proteus Schematic

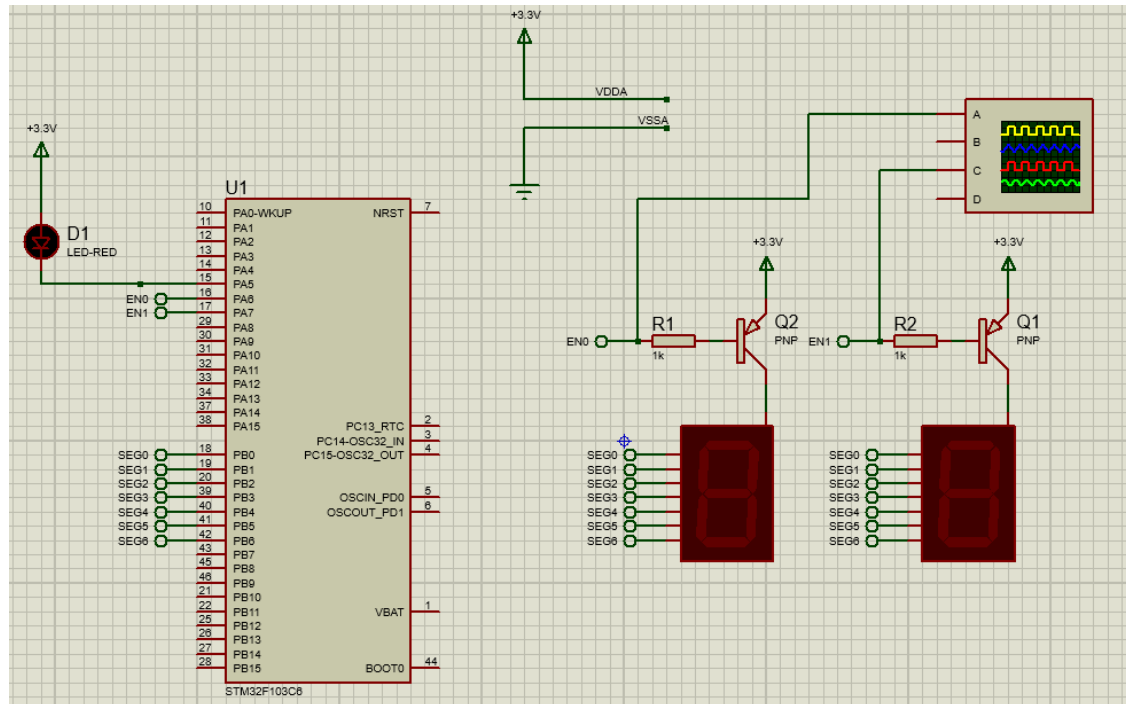


Figure 1.2: Schematic from Proteus for Exercise 1

STM32CubeIDE Configuration

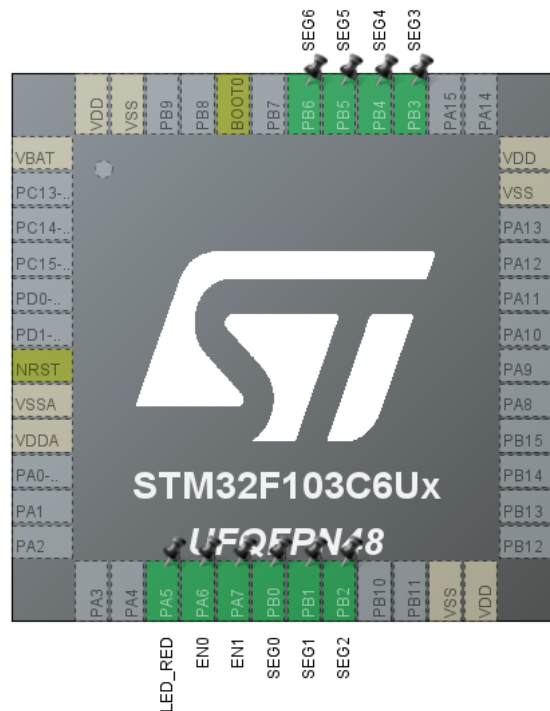


Figure 1.3: GPIO configuration for Exercise 1

STM32CubeIDE Code

```
1  /* USER CODE BEGIN PV */
2  int timer_counter = 50; // 50 * 10ms = 500ms delay
3  int led_state = 0;      // State variable: 0 for 1st LED, 1 for
                           // 2nd LED
4  /* USER CODE END PV */
5
6  /* USER CODE BEGIN PFP */
7  void display7SEG(int num);
8  /* USER CODE END PFP */
9
10 /* USER CODE BEGIN 2 */
11 HAL_TIM_Base_Start_IT(&htim2);
12 /* USER CODE END 2 */
13
14 /* USER CODE BEGIN 4 */
15 void display7SEG(int num) {
16     uint16_t seg_pins[7] = {
17         SEG0_Pin, SEG1_Pin, SEG2_Pin,
18         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
19     }; // PB0-PB6
20 }
```

```

21     uint8_t seg_data[10][7] = {
22         {0,0,0,0,0,0,1}, // 0
23         {1,0,0,1,1,1,1}, // 1
24         {0,0,1,0,0,1,0}, // 2
25         {0,0,0,0,1,1,0}, // 3
26         {1,0,0,1,1,0,0}, // 4
27         {0,1,0,0,1,0,0}, // 5
28         {0,1,0,0,0,0,0}, // 6
29         {0,0,0,1,1,1,1}, // 7
30         {0,0,0,0,0,0,0}, // 8
31         {0,0,0,0,1,0,0} // 9
32     };
33
34     if (num >= 0 && num <= 9) {
35         for (int i = 0; i < 7; i++) {
36             if (seg_data[num][i] == 0) {
37                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
38                 GPIO_PIN_RESET);
39             }
39             else {
40                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
41                 GPIO_PIN_SET);
42             }
43         }
44     }
45
46 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
47     if (htim->Instance == TIM2) {
48         timer_counter--;
49
50         if (timer_counter <= 0) {
51             timer_counter = 50; // Reset 500ms timer (50 * 10ms
52             interrupt period)
53
54             if (led_state == 0) {
55                 // Turn off 2nd 7SEG (EN1) before setting
56                 segments
57                 HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
58                 GPIO_PIN_SET);
59
60                 // Set segments for '1' and enable 1st 7SEG (
61                 EN0 - Active LOW for PNP)
62                 display7SEG(1);
63                 HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
64                 GPIO_PIN_RESET);
65
66                 led_state = 1;
67             }
68         }
69     }
70 }

```

```

63         else {
64             // Turn off 1st 7SEG (EN0) before setting
segments
65             HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
GPIO_PIN_SET);
66
67             // Set segments for '2' and enable 2nd 7SEG (
EN1 - Active LOW for PNP)
68             display7SEG(2);
69             HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
GPIO_PIN_RESET);
70
71             led_state = 0;
72         }
73     }
74 }
75 }
76 /* USER CODE END 4 */

```

Program 1.1: Source code for Exercise 1

1.2 Exercise 2

Extend to 4 seven segment LEDs and two LEDs(connected to PA4,labeled as DOT) in the middle as following:

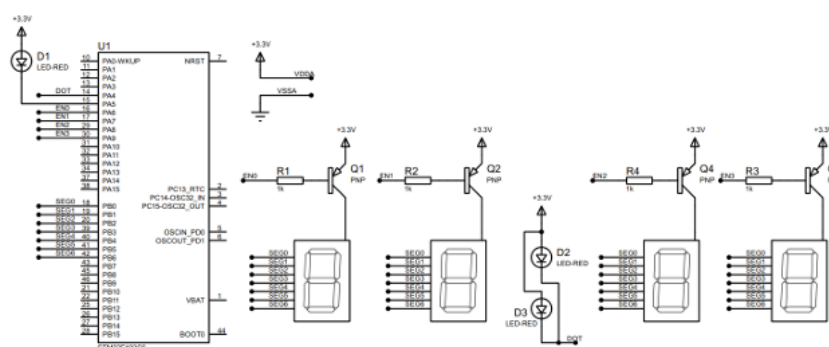


Figure 1.4: Simulation schematic in Proteus

Blink the two LEDs every second. Meanwhile, number 3 is displayed on the third seven segment and number 0 is displayed on the last one (to present 12 hour and a half). The switching time for each seven segment LED is also a half of second (500ms). **Implement your code in the timer interrupt function.**

Report 1: Capture your schematic from Proteus and show in the report.

Report 2: Present your source code in the `HAL_TIM_PeriodElapsedCallback` function.

Short question: What is the frequency of the scanning process?

Each 7-segment LED is switched every 0.5 seconds. Since there are 4 LEDs, a complete scanning cycle takes 2 seconds. Therefore, the scanning frequency is 0.5 Hz.

Proteus Schematic

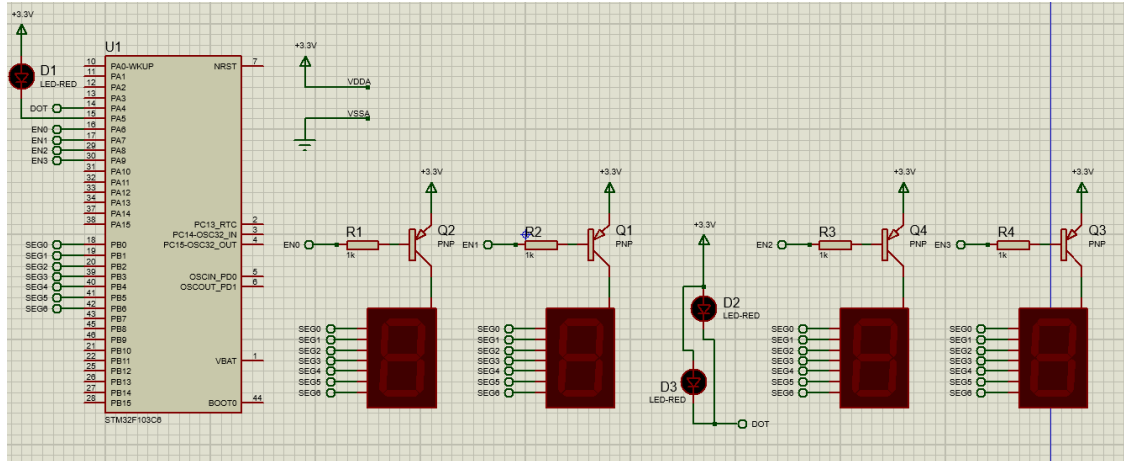


Figure 1.5: Schematic from Proteus for Exercise 2

STM32CubeIDE Configuration

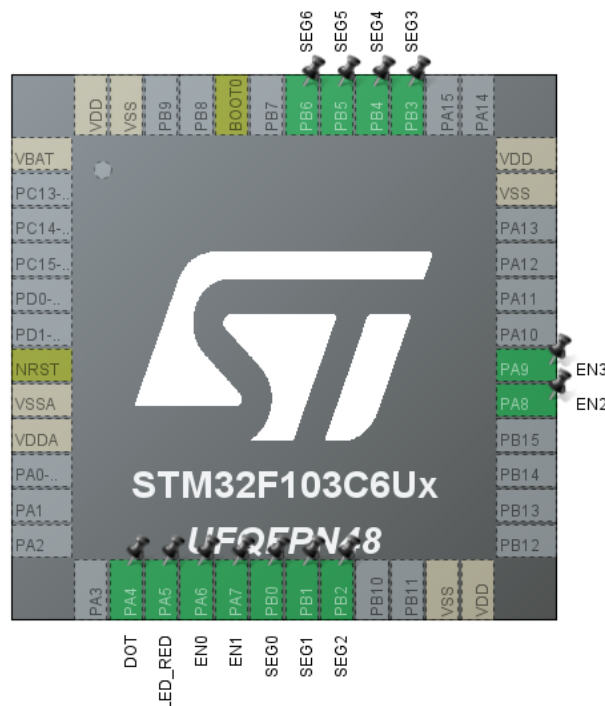


Figure 1.6: GPIO configuration for Exercise 2

STM32CubeIDE Code

```

1  /* USER CODE BEGIN PV */
2  int timer_7seg = 50;  // 50 * 10ms = 500ms delay for 7-segment
   display scanning
3  int timer_dot = 100;  // 100 * 10ms = 1000ms (1s) delay for DOT
   LEDs
4  int led_index = 0;    // Index of the currently active 7-
   segment LED (0 to 3)
5  /* USER CODE END PV */
6
7  /* USER CODE BEGIN PFP */
8  void display7SEG(int num);
9  void turnOffAll7SEG(void);
10 /* USER CODE END PFP */
11
12 /* USER CODE BEGIN 2 */
13 HAL_TIM_Base_Start_IT(&htim2);
14 /* USER CODE END 2 */
15
16 /* USER CODE BEGIN 4 */
17
18 void display7SEG(int num) {
19     uint16_t seg_pins[7] = {
20         SEG0_Pin, SEG1_Pin, SEG2_Pin,
21         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
22     }; // PB0 - PB6
23
24     uint8_t seg_data[10][7] = {
25         {0,0,0,0,0,0,1}, // 0
26         {1,0,0,1,1,1,1}, // 1
27         {0,0,1,0,0,1,0}, // 2
28         {0,0,0,0,1,1,0}, // 3
29         {1,0,0,1,1,0,0}, // 4
30         {0,1,0,0,1,0,0}, // 5
31         {0,1,0,0,0,0,0}, // 6
32         {0,0,0,1,1,1,1}, // 7
33         {0,0,0,0,0,0,0}, // 8
34         {0,0,0,0,1,0,0}  // 9
35     };
36
37     if (num >= 0 && num <= 9) {
38         for (int i = 0; i < 7; i++) {
39             if (seg_data[num][i] == 0) {
40                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
41             } else {
42                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
43             }
44         }

```

```

45     }
46 }
47
48 // Function to disable all 4 7-segment displays (SET = OFF for
49 // PNP transistor)
50 void turnOffAll7SEG(void) {
51     HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
52         EN3_Pin, GPIO_PIN_SET);
53 }
54
55 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
56     if (htim->Instance == TIM2) {
57         timer_dot--;
58
59         // 1. Toggle DOT LEDs (PA4) every 1 second (1000ms)
60         if (timer_dot <= 0) {
61             timer_dot = 100; // Reset 1000ms counter
62             HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
63         }
64
65         // 2. Scan 4 7-segment LEDs every 500ms
66         timer_7seg--;
67         if (timer_7seg <= 0) {
68             timer_7seg = 50; // Reset 500ms counter
69
70             // Turn off all displays to prevent ghosting
71             turnOffAll7SEG();
72
73             // Display corresponding digit and activate
74             // corresponding Enable pin
75             switch (led_index) {
76                 case 0:
77                     display7SEG(1);
78                     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
79                         GPIO_PIN_RESET);
80                     break;
81                 case 1:
82                     display7SEG(2);
83                     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
84                         GPIO_PIN_RESET);
85                     break;
86                 case 2:
87                     display7SEG(3);
88                     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
89                         GPIO_PIN_RESET);
90                     break;
91                 case 3:
92                     display7SEG(0);
93                     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,

```

```

GPIO_PIN_RESET);
88         break;
89     default:
90         break;
91     }
92
93     // Move to the next LED index
94     led_index++;
95     if (led_index >= 4) {
96         led_index = 0;
97     }
98 }
99 }
100 }
101 /* USER CODE END 4 */

```

Program 1.2: Source code for Exercise 2

1.3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

STM32CubeIDE Code Skeleton

```

1  const int MAX_LED = 4;
2  int index_led = 0;
3  int led_buffer[4] = {1, 2, 3, 4};
4  void update7SEG(int index){
5      switch (index){
6          case 0:
7              //Display the first 7SEG with led_buffer[0]
8              break;
9          case 1:
10             //Display the second 7SEG with led_buffer[1]
11             break;
12          case 2:
13             //Display the third 7SEG with led_buffer[2]
14             break;
15          case 3:
16             //Display the forth 7SEG with led_buffer[3]
17             break;
18          default:
19             break;
20      }
21 }

```

Program 1.3: An example for your source code

This function should be invoked in the timer interrupt, e.g `update7SEG(indexled++)`. The variable

Report1: Present the source code of the `update7SEG` function.

Report2: Present the source code in the `HAL_TIM_PeriodElapsedCallback`

Students are proposed to change the values in the `led_buffer` array for unit test this function, which is used afterward.

Proteus Schematic

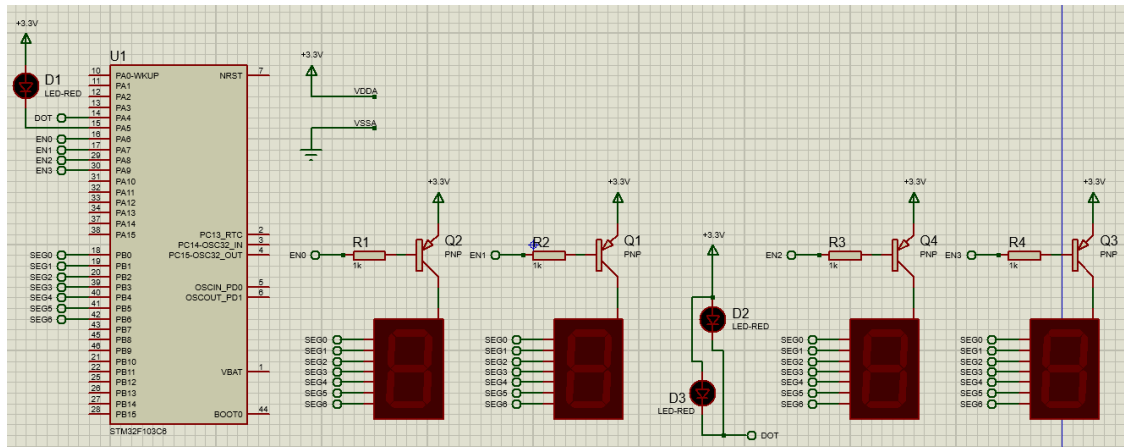


Figure 1.7: Schematic from Proteus for Exercise 3

STM32CubeIDE Configuration

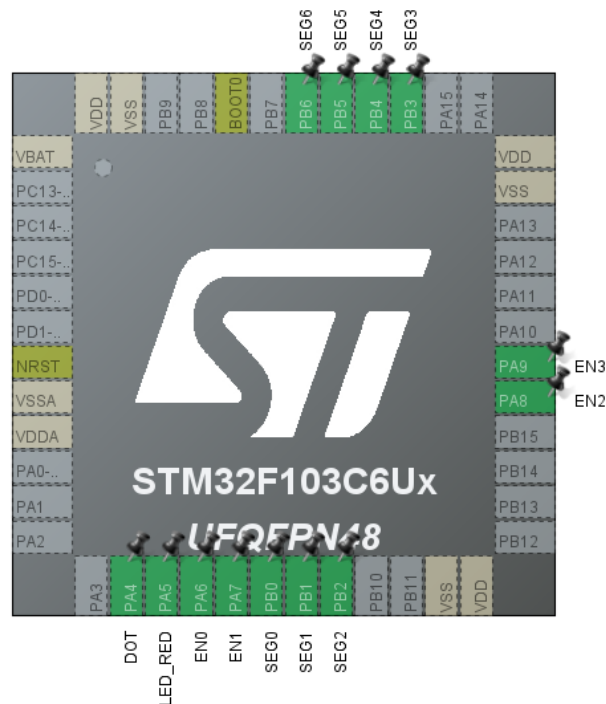


Figure 1.8: GPIO configuration for Exercise 3

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
2 const int MAX_LED = 4;
3 int index_led = 0;
4 int led_buffer[4] = {1, 2, 3, 0}; // Buffer array holding
   values to display "12:30"
5
6 int timer_7seg = 50; // 50 * 10ms = 500ms scanning delay per
   LED (1s for full cycle)
7 int timer_dot = 100; // 100 * 10ms = 1000ms (1s) delay for DOT
   LEDs
8 /* USER CODE END PV */
9
10 /* USER CODE BEGIN PFP */
11 void display7SEG(int num);
12 void update7SEG(int index);
13 /* USER CODE END PFP */
14
15 /* USER CODE BEGIN 2 */
16 HAL_TIM_Base_Start_IT(&htim2);
17 /* USER CODE END 2 */
18
```

```

19 void display7SEG(int num) {
20     uint16_t seg_pins[7] = {
21         SEG0_Pin, SEG1_Pin, SEG2_Pin,
22         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
23     }; // PB0 - PB6
24
25     uint8_t seg_data[10][7] = {
26         {0,0,0,0,0,0,1}, // 0
27         {1,0,0,1,1,1,1}, // 1
28         {0,0,1,0,0,1,0}, // 2
29         {0,0,0,0,1,1,0}, // 3
30         {1,0,0,1,1,0,0}, // 4
31         {0,1,0,0,1,0,0}, // 5
32         {0,1,0,0,0,0,0}, // 6
33         {0,0,0,1,1,1,1}, // 7
34         {0,0,0,0,0,0,0}, // 8
35         {0,0,0,0,1,0,0} // 9
36     };
37
38     if (num >= 0 && num <= 9) {
39         for (int i = 0; i < 7; i++) {
40             if (seg_data[num][i] == 0) {
41                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
42             } else {
43                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
44             }
45         }
46     }
47 }
48
49 // Function to update a specific 7-segment display based on
index
50 void update7SEG(int index) {
51     // 1. Disable all 4 displays first to prevent ghosting (SET
= OFF for PNP)
52     HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
EN3_Pin, GPIO_PIN_SET);
53
54     // 2. Display value from led_buffer[index] and turn on the
corresponding EN pin
55     switch (index) {
56         case 0:
57             display7SEG(led_buffer[0]);
58             HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
GPIO_PIN_RESET);
59             break;
60         case 1:

```

```

61         display7SEG(led_buffer[1]);
62         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
GPIO_PIN_RESET);
63         break;
64     case 2:
65         display7SEG(led_buffer[2]);
66         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
GPIO_PIN_RESET);
67         break;
68     case 3:
69         display7SEG(led_buffer[3]);
70         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
GPIO_PIN_RESET);
71         break;
72     default:
73         break;
74 }
75 }
76
77 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
78     if (htim->Instance == TIM2) {
79
80         // 1. Toggle DOT LEDs (PA4) every 1 second (1000ms)
81         timer_dot--;
82         if (timer_dot <= 0) {
83             timer_dot = 100; // Reset 1000ms counter
84             HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
85         }
86
87         // 2. Scan 7-segment display using update7SEG function
88         timer_7seg--;
89         if (timer_7seg <= 0) {
90             timer_7seg = 50; // Reset scanning counter (500ms)
91
92             // Update display at current index
93             update7SEG(index_led);
94
95             // Increment index and roll over at MAX_LED
96             index_led++;
97             if (index_led >= MAX_LED) {
98                 index_led = 0;
99             }
100         }
101     }
102 }

```

Program 1.4: Source code for Exercise 3

1.4 Exercise 4

Change the period of invoking update7SEG function in order to set the frequency of 4 seven segment LEDs to 1 Hz. The DOT is still blinking every second.

Report 1: Present the source code in the `HAL_TIM_PeriodElapsedCallback`.

Proteus Schematic

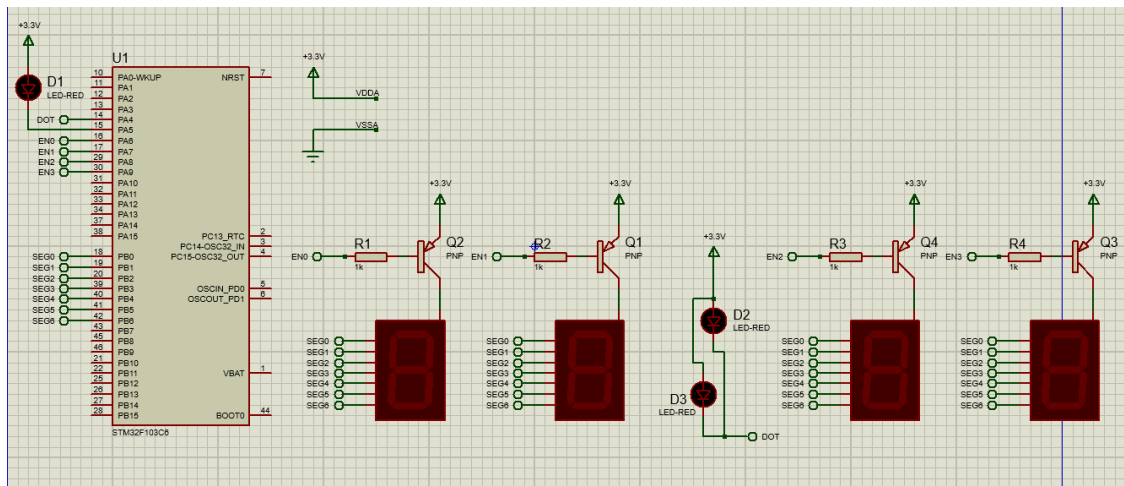


Figure 1.9: Schematic from Proteus for Exercise 4

STM32CubeIDE Configuration

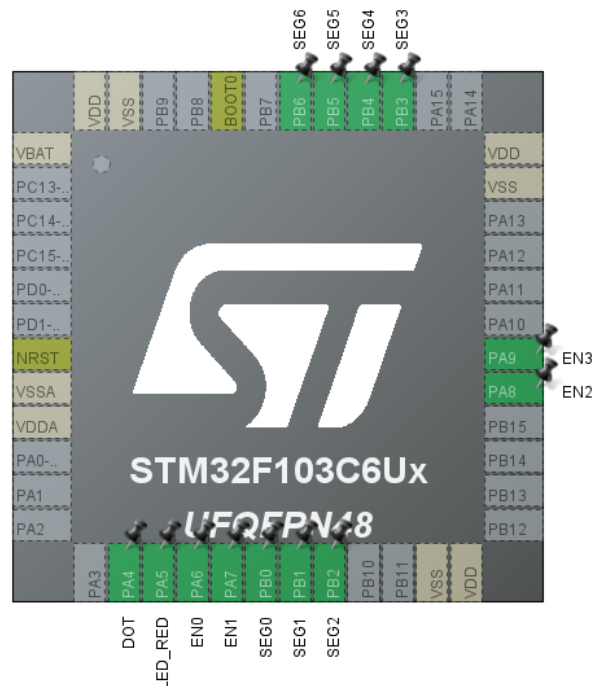


Figure 1.10: GPIO configuration for Exercise 4

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
2 const int MAX_LED = 4;
3 int index_led = 0;
4 int led_buffer[4] = {1, 2, 3, 0}; // Buffer array holding
   values to display "12:30"
5
6 int timer_7seg = 25; // 25 * 10ms = 250ms scanning delay per
   LED (1s for full cycle)
7 int timer_dot = 100; // 100 * 10ms = 1000ms (1s) delay for DOT
   LEDs
8 /* USER CODE END PV */
9
10 /* USER CODE BEGIN PFP */
11 void display7SEG(int num);
12 void update7SEG(int index);
13 /* USER CODE END PFP */
14
15 /* USER CODE BEGIN 2 */
16 HAL_TIM_Base_Start_IT(&htim2);
17 /* USER CODE END 2 */
18
```

```

19 /* USER CODE BEGIN 4 */
20 void display7SEG(int num) {
21     uint16_t seg_pins[7] = {
22         SEG0_Pin, SEG1_Pin, SEG2_Pin,
23         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
24     }; // PB0 - PB6
25
26     uint8_t seg_data[10][7] = {
27         {0,0,0,0,0,0,1}, // 0
28         {1,0,0,1,1,1,1}, // 1
29         {0,0,1,0,0,1,0}, // 2
30         {0,0,0,0,1,1,0}, // 3
31         {1,0,0,1,1,0,0}, // 4
32         {0,1,0,0,1,0,0}, // 5
33         {0,1,0,0,0,0,0}, // 6
34         {0,0,0,1,1,1,1}, // 7
35         {0,0,0,0,0,0,0}, // 8
36         {0,0,0,0,1,0,0} // 9
37     };
38
39     if (num >= 0 && num <= 9) {
40         for (int i = 0; i < 7; i++) {
41             if (seg_data[num][i] == 0) {
42                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
43             } else {
44                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
45             }
46         }
47     }
48 }
49
50 // Function to update a specific 7-segment display based on
index
51 void update7SEG(int index) {
52     // 1. Disable all 4 displays first to prevent ghosting (SET
= OFF for PNP)
53     HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin |
EN3_Pin, GPIO_PIN_SET);
54
55     // 2. Display value from led_buffer[index] and turn on the
corresponding EN pin
56     switch (index) {
57         case 0:
58             display7SEG(led_buffer[0]);
59             HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
GPIO_PIN_RESET);
60             break;

```

```

61     case 1:
62         display7SEG(led_buffer[1]);
63         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
GPIO_PIN_RESET);
64         break;
65     case 2:
66         display7SEG(led_buffer[2]);
67         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
GPIO_PIN_RESET);
68         break;
69     case 3:
70         display7SEG(led_buffer[3]);
71         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
GPIO_PIN_RESET);
72         break;
73     default:
74         break;
75 }
76 }
77
78 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
79     if (htim->Instance == TIM2) {
80
81         // 1. Toggle DOT LEDs (PA4) every 1 second (1000ms)
82         timer_dot--;
83         if (timer_dot <= 0) {
84             timer_dot = 100; // Reset 1000ms counter
85             HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
86         }
87
88         // 2. Scan 7-segment display using update7SEG function
89         timer_7seg--;
90         if (timer_7seg <= 0) {
91             timer_7seg = 25; // Reset scanning counter (250ms)
92
93             // Update display at current index
94             update7SEG(index_led);
95
96             // Increment index and roll over at MAX_LED
97             index_led++;
98             if (index_led >= MAX_LED) {
99                 index_led = 0;
100             }
101         }
102     }
103 }
104 /* USER CODE END 4 */

```

Program 1.5: Source code for Exercise 4

1.5 Exercise 5

Implement a digital clock with hour and minute information displayed by 2 seven segment LEDs. The code skeleton in the main function is presented as follows:

STM32CubeIDE Code

```
1 int hour = 15, minute = 8, second = 50;
2
3 while(1){
4     second++;
5     if (second >= 60){
6         second = 0;
7         minute++;
8     }
9     if(minute >= 60){
10        minute = 0;
11        hour++;
12    }
13    if(hour >=24){
14        hour = 0;
15    }
16    updateClockBuffer();
17    HAL_Delay(1000);
18 }
```

Program 1.6: An example for your source code

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

Report 1: Present the source code in the updateClockBuffer function.

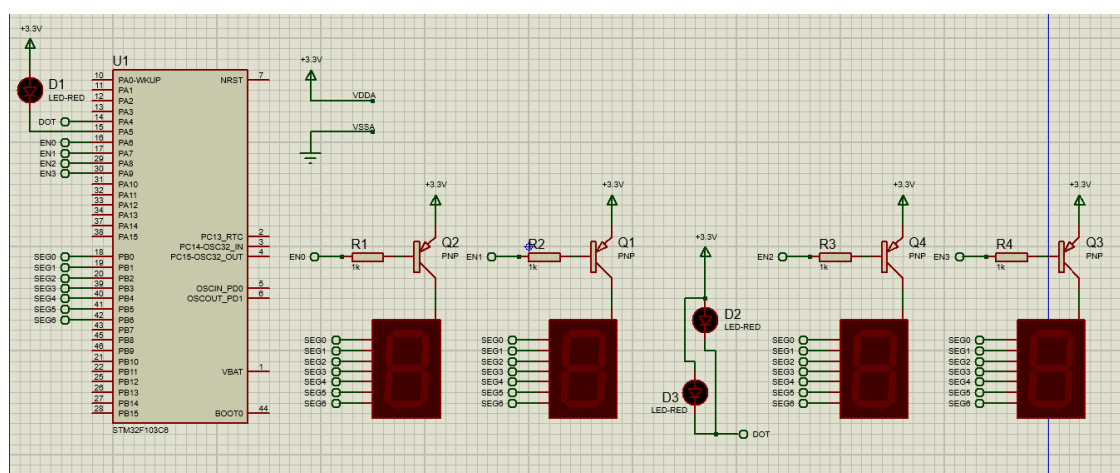


Figure 1.11: Schematic from Proteus for Exercise 5

STM32CubeIDE Configuration

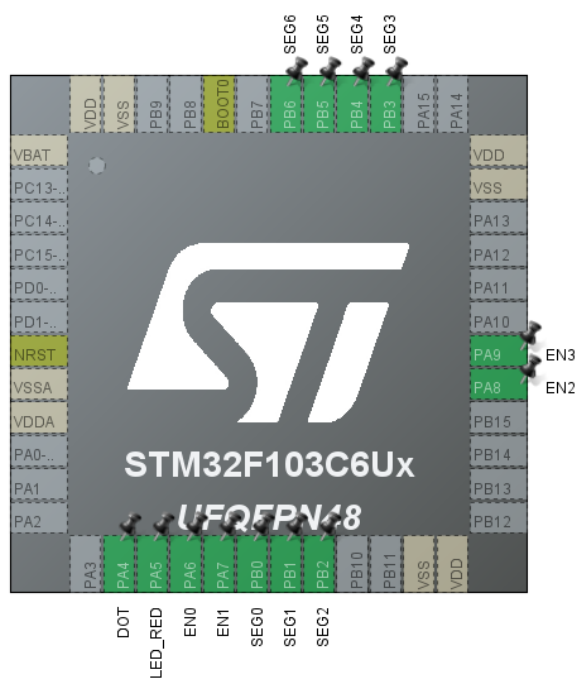


Figure 1.12: GPIO configuration for Exercise 5

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
2 const int MAX_LED = 4;
3 int index_led = 0;
```

```

4 int led_buffer[4] = {1, 5, 0, 8}; // Initial values for display
   buffer
5
6 // Clock variables initialized to 15:08:50
7 int hour = 15;
8 int minute = 8;
9 int second = 50;
10
11 // Software timer counter for display scan and clock tick in
   Exercise 5
12 int timer_7seg = 25; // 25 * 10ms = 250ms scanning delay per
   LED (1s for full cycle)
13 int timer_dot = 100; // 1000ms (1s)
14 /* USER CODE END PV */
15
16 /* USER CODE BEGIN PFP */
17 void display7SEG(int num);
18 void update7SEG(int index);
19 void updateClockBuffer(void);
20 /* USER CODE END PFP */
21
22 /* USER CODE BEGIN 2 */
23 // Update display buffer once before starting timer interrupts
24 updateClockBuffer();
25
26 // Start Timer 2 in interrupt mode
27 HAL_TIM_Base_Start_IT(&htim2);
28 /* USER CODE END 2 */
29
30 /* USER CODE BEGIN WHILE */
31 while (1)
32 {
33     second++;
34
35     if (second >= 60)
36     {
37         second = 0;
38         minute++;
39     }
40
41     if (minute >= 60)
42     {
43         minute = 0;
44         hour++;
45     }
46
47     if (hour >= 24)
48     {
49         hour = 0;

```

```

50     }
51
52     updateClockBuffer();
53
54     HAL_Delay(1000);
55     /* USER CODE END WHILE */
56
57     /* USER CODE BEGIN 3 */
58 }
59 /* USER CODE END 3 */
60
61 /* USER CODE BEGIN 4 */
62 void updateClockBuffer(void)
63 {
64     led_buffer[0] = hour / 10;
65     led_buffer[1] = hour % 10;
66     led_buffer[2] = minute / 10;
67     led_buffer[3] = minute % 10;
68 }
69
70 void display7SEG(int num) {
71     uint16_t seg_pins[7] = {
72         SEG0_Pin, SEG1_Pin, SEG2_Pin,
73         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
74     };
75
76     uint8_t seg_data[10][7] = {
77         {0,0,0,0,0,0,1}, // 0
78         {1,0,0,1,1,1,1}, // 1
79         {0,0,1,0,0,1,0}, // 2
80         {0,0,0,0,1,1,0}, // 3
81         {1,0,0,1,1,0,0}, // 4
82         {0,1,0,0,1,0,0}, // 5
83         {0,1,0,0,0,0,0}, // 6
84         {0,0,0,1,1,1,1}, // 7
85         {0,0,0,0,0,0,0}, // 8
86         {0,0,0,0,1,0,0}  // 9
87     };
88
89     if (num >= 0 && num <= 9) {
90         for (int i = 0; i < 7; i++) {
91             if (seg_data[num][i] == 0) {
92                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
93                 GPIO_PIN_RESET);
94             } else {
95                 HAL_GPIO_WritePin(GPIOB, seg_pins[i], GPIO_PIN_SET)
96             }
97         }
98     }
99 }

```

```

97 }
98 }
99
100 void update7SEG(int index) {
101     switch (index) {
102         case 0:
103             display7SEG(led_buffer[0]);
104             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
105             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
106             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
107             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
108             break;
109         case 1:
110             display7SEG(led_buffer[1]);
111             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
112             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
113             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
114             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
115             break;
116         case 2:
117             display7SEG(led_buffer[2]);
118             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
119             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
120             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
121             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
122             break;
123         case 3:
124             display7SEG(led_buffer[3]);
125             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
126             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
127             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
128             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);
129             break;
130         default:
131             break;
132     }
133 }
134
135 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
136     if (htim->Instance == TIM2) {
137
138         // 1. Toggle DOT LEDs (PA4) every 1 second (1000ms)
139         timer_dot--;
140         if (timer_dot <= 0) {
141             timer_dot = 100; // Reset 1000ms counter
142             HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
143         }
144
145         // 2. Scan 7-segment display using update7SEG function

```

```

146     timer_7seg--;
147     if (timer_7seg <= 0) {
148         timer_7seg = 25; // Reset scanning counter (250ms)
149
150         // Update display at current index
151         update7SEG(index_led);
152
153         // Increment index and roll over at MAX_LED
154         index_led++;
155         if (index_led >= MAX_LED) {
156             index_led = 0;
157         }
158     }
159 }
160 }
161
162 /* USER CODE END 4 */

```

Program 1.7: Source code for Exercise 5

1.6 Exercise 6

The main target from this exercise is to reduce the complexity (or reduce code processing) in the timer interrupt. The time consumed in the interrupt can lead to the nested interrupt issue, which can crash the whole system. A simple solution can disable the timer whenever the interrupt occurs, then enable it again. However, the real-time processing is not guaranteed anymore.

In this exercise, a software timer is created and its counter is counted down every timer interrupt is raised (every 10ms). By using this timer, the `HAL_Delay(1000)` in the main function is removed. In a MCUs system, non-blocking delay is better than blocking delay. The details to create a software timer are presented below. The source code is added to your current program, do not delete the source code you have on Exercise 5.

Step 1: Declare variables and functions for a software timer, as following:

```

1  /* USER CODE BEGIN 0 */
2  int timer0_counter = 0;
3  int timer0_flag = 0;
4  int TIMER_CYCLE = 10;
5
6  void setTimer0(int duration){
7      timer0_counter = duration / TIMER_CYCLE;
8      timer0_flag = 0;
9  }
10
11 void timer_run(){
12     if(timer0_counter > 0){
13         timer0_counter--;
14         if(timer0_counter == 0) timer0_flag = 1;

```

```

15     }
16 }
17 /* USER CODE END 0 */

```

Program 1.8: Software timer based timer interrupt

Please change the **TIMER_CYCLE** to your timer interrupt period. In the manual code above, it is **10ms**. **Step 2:** The **timer_run()** is invoked in the timer interrupt as following:

```

1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     timer_run();
4     // YOUR OTHER CODE
5 }

```

Program 1.9: Software timer based timer interrupt

Step 3: Use the timer in the main function by invoking the **setTimer0** function, then check for its flag (**timer0_flag**). An example to blink an LED connected to PA5 using software timer is shown as follows:

```

1 setTimer0(1000);
2
3 while (1){
4     if(timer0_flag == 1){
5         HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
6         setTimer0(2000);
7     }
8 }

```

Program 1.10: Software timer is used in main function to blink the LED

Report 1: If in line 1 of the code above is missed, what happens after that and why?

If line 1 is missed, **timer0_counter** is never initialized with a duration, so it remains 0. Therefore, **timer0_flag** will never become 1 and the LED will not toggle.

Report 2: If in line 1 of the code above is changed to **setTimer0(1)**, what happens after that and why?

The LED will not toggle because $1 / \text{TIMER_CYCLE} = 1 / 10 = 0$, so **timer0_counter** becomes 0 immediately and **timer0_flag** never becomes 1.

Report 3: If in line 1 of the code above is changed to **setTimer0(10)**, what is changed compared to the first two questions and why?

When **setTimer0(10)** is used, **timer0_counter** is calculated as $10 / 10 = 1$. Since the timer interrupt occurs every 10 ms, the counter becomes zero after one interrupt, and **timer0_flag** is set to 1. Therefore, the LED toggles after 10 ms. This is different from **setTimer0(1)**, where integer division makes **timer0_counter** equal to 0, so the flag is never set. After the first toggle, **setTimer0(2000)** is called, so the LED toggles every 2 seconds thereafter.

Proteus Schematic

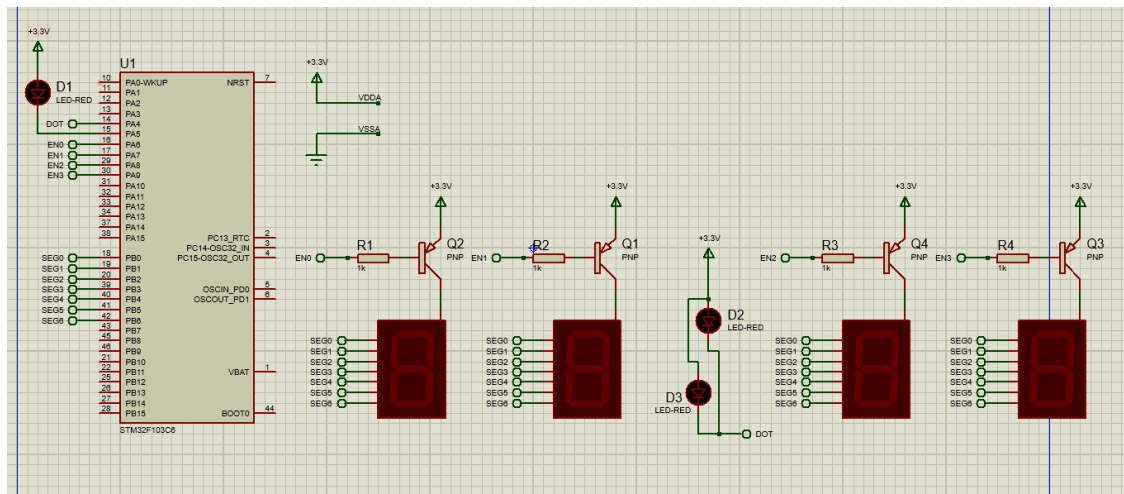


Figure 1.13: Schematic from Proteus for Exercise 6

STM32CubeIDE Configuration

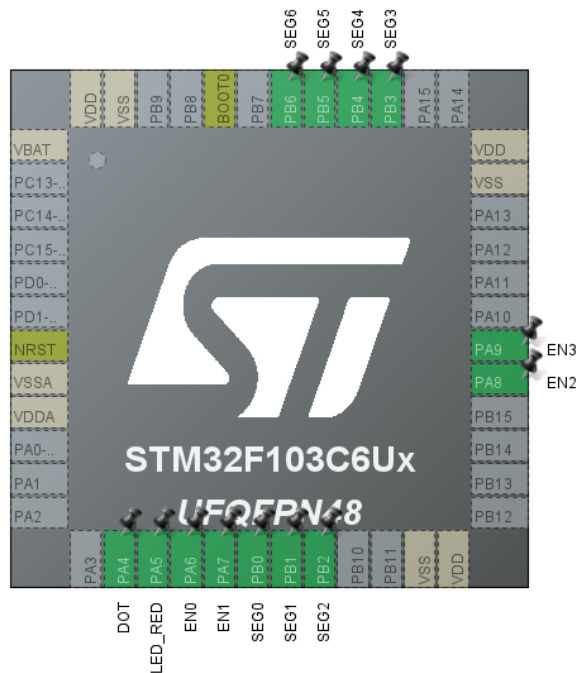


Figure 1.14: GPIO configuration for Exercise 6

```
1 /* USER CODE BEGIN PV */
2 const int MAX_LED = 4;
3 int index_led = 0;
4 int led_buffer[4] = {1, 5, 0, 8}; // Initial values for display
   buffer
```

```

5
6 // Clock variables initialized to 15:08:50
7 int hour = 15;
8 int minute = 8;
9 int second = 50;
10
11 // Software timer counter for display scan and clock tick in
    Exercise 5
12 int timer_7seg = 25; // 25 * 10ms = 250ms scanning delay per
    LED (1s for full cycle)
13 int timer_dot = 100; // 1000ms (1s)
14 /* USER CODE END PV */
15
16 /* USER CODE BEGIN PFP */
17 void display7SEG(int num);
18 void update7SEG(int index);
19 void updateClockBuffer(void);
20 /* USER CODE END PFP */
21
22 /* USER CODE BEGIN 0 */
23 int timer0_counter = 0;
24 int timer0_flag = 0;
25 int TIMER_CYCLE = 10;
26 void setTimer0(int duration) {
27     timer0_counter = duration / TIMER_CYCLE;
28     timer0_flag = 0;
29 }
30 void timer_run(){
31     if (timer0_counter > 0) {
32         timer0_counter--;
33         if (timer0_counter == 0) timer0_flag = 1;
34     }
35 }
36 /* USER CODE END 0 */
37
38 /* USER CODE BEGIN 2 */
39 // Update display buffer once before starting timer interrupts
40 updateClockBuffer();
41
42 // Start Timer 2 in interrupt mode
43 HAL_TIM_Base_Start_IT(&htim2);
44 /* USER CODE END 2 */
45
46 /* USER CODE BEGIN WHILE */
47 setTimer0(1000);
48 while (1)
49 {
50     if (timer0_flag == 1) {
51         second++;

```

```

52
53     if (second >= 60)
54     {
55         second = 0;
56         minute++;
57     }
58
59     if (minute >= 60)
60     {
61         minute = 0;
62         hour++;
63     }
64
65     if (hour >= 24)
66     {
67         hour = 0;
68     }
69
70     updateClockBuffer();
71
72     setTimer0(1000);
73 }
74 /* USER CODE END WHILE */
75
76 /* USER CODE BEGIN 3 */
77 }
78 /* USER CODE END 3 */
79
80 /* USER CODE BEGIN 4 */
81 void updateClockBuffer(void)
82 {
83     led_buffer[0] = hour / 10;
84     led_buffer[1] = hour % 10;
85     led_buffer[2] = minute / 10;
86     led_buffer[3] = minute % 10;
87 }
88
89 void display7SEG(int num) {
90     uint16_t seg_pins[7] = {
91         SEG0_Pin, SEG1_Pin, SEG2_Pin,
92         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
93     };
94
95     uint8_t seg_data[10][7] = {
96         {0,0,0,0,0,0,1}, // 0
97         {1,0,0,1,1,1,1}, // 1
98         {0,0,1,0,0,1,0}, // 2
99         {0,0,0,0,1,1,0}, // 3
100        {1,0,0,1,1,0,0}, // 4

```

```

101     {0,1,0,0,1,0,0}, // 5
102     {0,1,0,0,0,0,0}, // 6
103     {0,0,0,1,1,1,1}, // 7
104     {0,0,0,0,0,0,0}, // 8
105     {0,0,0,0,1,0,0} // 9
106 };
107
108     if (num >= 0 && num <= 9) {
109         for (int i = 0; i < 7; i++) {
110             if (seg_data[num][i] == 0) {
111                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
112             } else {
113                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
114             }
115         }
116     }
117 }
118
119 void update7SEG(int index) {
120     switch (index) {
121         case 0:
122             display7SEG(led_buffer[0]);
123             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
124             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
125             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
126             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
127             break;
128         case 1:
129             display7SEG(led_buffer[1]);
130             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
131             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
132             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
133             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
134             break;
135         case 2:
136             display7SEG(led_buffer[2]);
137             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
138             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
139             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
140             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
141             break;
142         case 3:
143             display7SEG(led_buffer[3]);
144             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
145             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
146             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
147             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);

```

```

148         break;
149     default:
150         break;
151 }
152 }
153
154 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
155     if (htim->Instance == TIM2) {
156         timer_run();
157
158         // 1. Toggle DOT LEDs (PA4) every 1 second (1000ms)
159         timer_dot--;
160         if (timer_dot <= 0) {
161             timer_dot = 100; // Reset 1000ms counter
162             HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
163         }
164
165         // 2. Scan 7-segment display using update7SEG function
166         timer_7seg--;
167         if (timer_7seg <= 0) {
168             timer_7seg = 25; // Reset scanning counter (250ms)
169
170             // Update display at current index
171             update7SEG(index_led);
172
173             // Increment index and roll over at MAX_LED
174             index_led++;
175             if (index_led >= MAX_LED) {
176                 index_led = 0;
177             }
178         }
179     }
180 }
181
182 /* USER CODE END 4 */

```

Program 1.11: Source code for Exercise 6

1.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present your source code in the while loop on main function.

Proteus Schematic

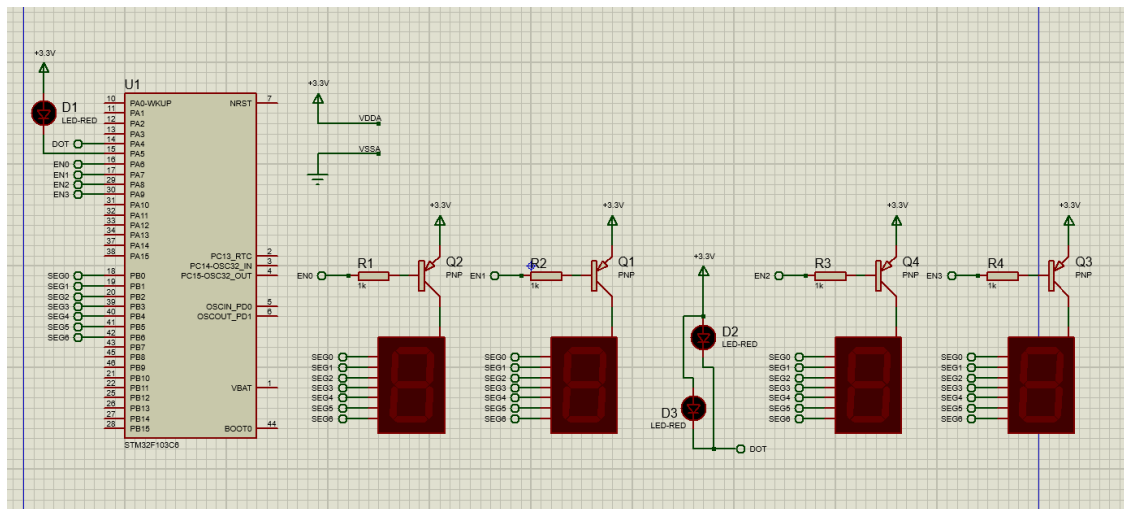


Figure 1.15: Schematic from Proteus for Exercise 7

STM32CubeIDE Configuration

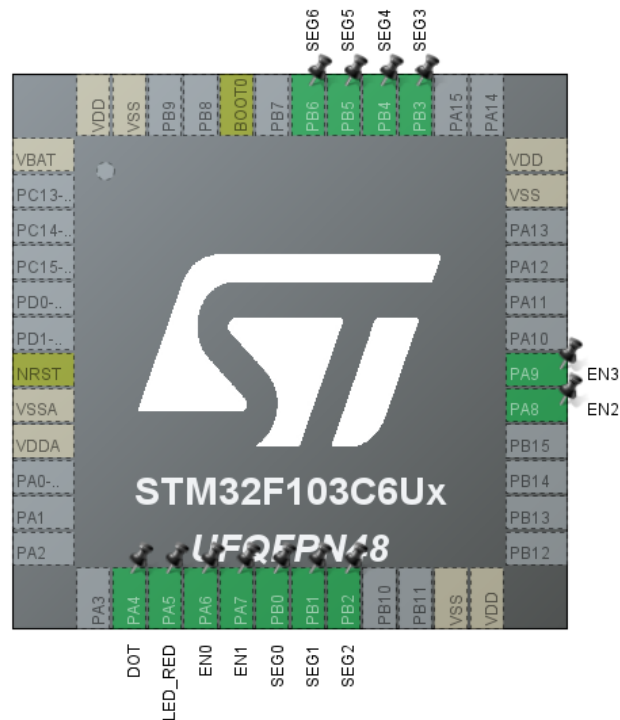


Figure 1.16: GPIO configuration for Exercise 7

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
```

```

2 const int MAX_LED = 4;
3 int index_led = 0;
4 int led_buffer[4] = {1, 5, 0, 8}; // Initial values for display
    buffer
5
6 // Clock variables initialized to 15:08:50
7 int hour = 15;
8 int minute = 8;
9 int second = 50;
10
11 // Software timer counter for display scan and clock tick in
    Exercise 5
12 int timer_7seg = 25; // 25 * 10ms = 250ms scanning delay per
    LED (1s for full cycle)
13 /* USER CODE END PV */
14
15 /* USER CODE BEGIN PFP */
16 void display7SEG(int num);
17 void update7SEG(int index);
18 void updateClockBuffer(void);
19 /* USER CODE END PFP */
20
21 /* USER CODE BEGIN 0 */
22 int timer0_counter = 0;
23 int timer0_flag = 0;
24 int TIMER_CYCLE = 10;
25 void setTimer0(int duration) {
26     timer0_counter = duration / TIMER_CYCLE;
27     timer0_flag = 0;
28 }
29 void timer_run(){
30     if (timer0_counter > 0) {
31         timer0_counter--;
32         if (timer0_counter == 0) timer0_flag = 1;
33     }
34 }
35 /* USER CODE END 0 */
36
37 /* USER CODE BEGIN 2 */
38 // Update display buffer once before starting timer interrupts
39 updateClockBuffer();
40
41 // Start Timer 2 in interrupt mode
42 HAL_TIM_Base_Start_IT(&htim2);
43 /* USER CODE END 2 */
44
45 /* USER CODE BEGIN WHILE */
46 setTimer0(1000);
47 while (1)

```

```

48 {
49     if (timer0_flag == 1) {
50         second++;
51
52         if (second >= 60)
53         {
54             second = 0;
55             minute++;
56         }
57
58         if (minute >= 60)
59         {
60             minute = 0;
61             hour++;
62         }
63
64         if (hour >= 24)
65         {
66             hour = 0;
67         }
68
69         updateClockBuffer();
70
71         HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
72
73         setTimer0(1000);
74     }
75     /* USER CODE END WHILE */
76
77     /* USER CODE BEGIN 3 */
78 }
79
80 /* USER CODE BEGIN 4 */
81 void updateClockBuffer(void)
82 {
83     led_buffer[0] = hour / 10;
84     led_buffer[1] = hour % 10;
85     led_buffer[2] = minute / 10;
86     led_buffer[3] = minute % 10;
87 }
88
89 void display7SEG(int num) {
90     uint16_t seg_pins[7] = {
91         SEG0_Pin, SEG1_Pin, SEG2_Pin,
92         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
93     };
94
95     uint8_t seg_data[10][7] = {
96         {0,0,0,0,0,0,1}, // 0

```

```

97     {1,0,0,1,1,1,1}, // 1
98     {0,0,1,0,0,1,0}, // 2
99     {0,0,0,0,1,1,0}, // 3
100    {1,0,0,1,1,0,0}, // 4
101    {0,1,0,0,1,0,0}, // 5
102    {0,1,0,0,0,0,0}, // 6
103    {0,0,0,1,1,1,1}, // 7
104    {0,0,0,0,0,0,0}, // 8
105    {0,0,0,0,1,0,0} // 9
106 };
107
108     if (num >= 0 && num <= 9) {
109         for (int i = 0; i < 7; i++) {
110             if (seg_data[num][i] == 0) {
111                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
112             } else {
113                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
114             }
115         }
116     }
117 }
118
119 void update7SEG(int index) {
120     switch (index) {
121         case 0:
122             display7SEG(led_buffer[0]);
123             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
124             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
125             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
126             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
127             break;
128         case 1:
129             display7SEG(led_buffer[1]);
130             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
131             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
132             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
133             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
134             break;
135         case 2:
136             display7SEG(led_buffer[2]);
137             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
138             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
139             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
140             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
141             break;
142         case 3:
143             display7SEG(led_buffer[3]);

```

```

144     HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
145     HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
146     HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
147     HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);
148     break;
149     default:
150         break;
151 }
152 }
153
154 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
155     if (htim->Instance == TIM2) {
156         timer_run();
157
158         // Scan 7-segment display using update7SEG function
159         timer_7seg--;
160         if (timer_7seg <= 0) {
161             timer_7seg = 25; // Reset scanning counter (250ms)
162
163             // Update display at current index
164             update7SEG(index_led);
165
166             // Increment index and roll over at MAX_LED
167             index_led++;
168             if (index_led >= MAX_LED) {
169                 index_led = 0;
170             }
171         }
172     }
173 }
174
175 /* USER CODE END 4 */

```

Program 1.12: Source code for Exercise 7

1.8 Exercise 8

Move also the `update7SEG()` function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex computations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

Report 1: Present your source code in the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

Proteus Schematic

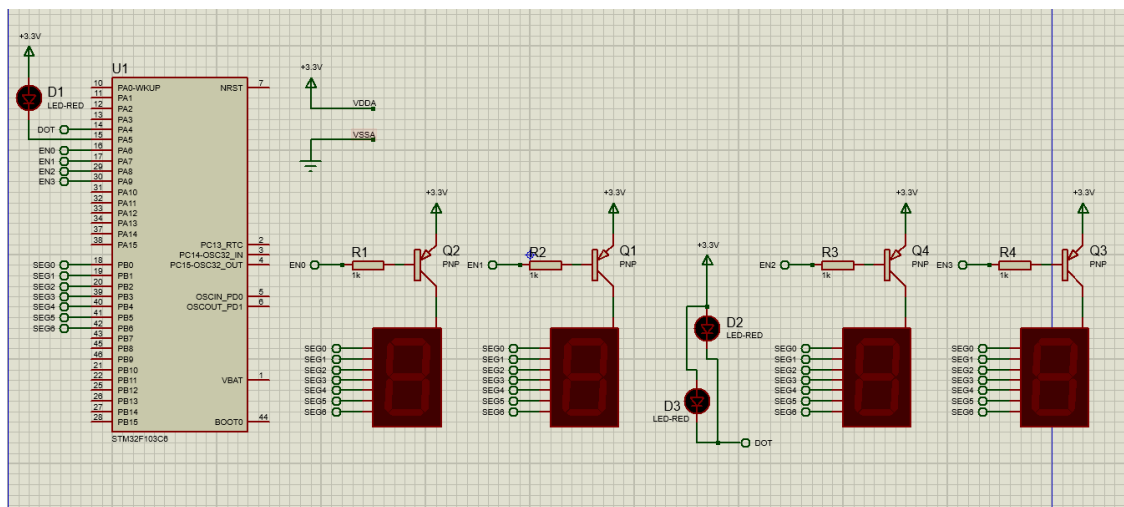


Figure 1.17: Schematic from Proteus for Exercise 8

STM32CubeIDE Configuration

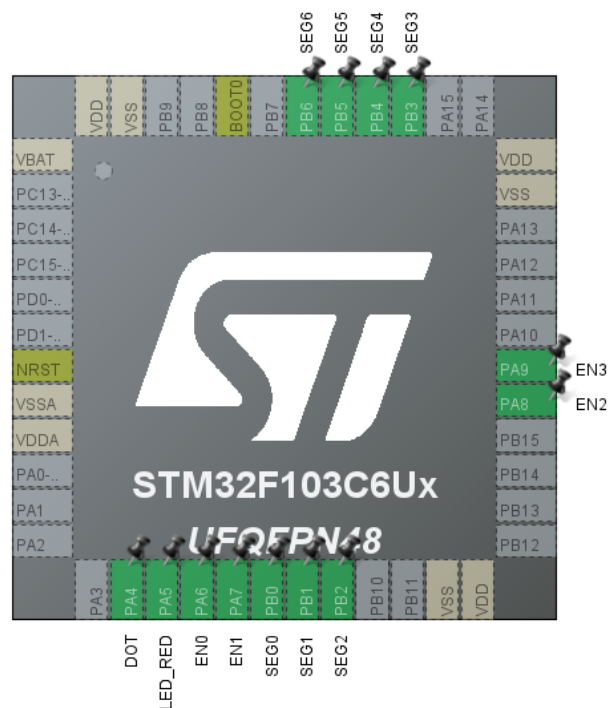


Figure 1.18: GPIO configuration for Exercise 8

STM32CubeIDE Code

```

1  /* USER CODE BEGIN PV */
2  const int MAX_LED = 4;
3  int index_led = 0;
4  int led_buffer[4] = {1, 5, 0, 8}; // Initial values for display
    buffer
5
6  // Clock variables initialized to 15:08:50
7  int hour = 15;
8  int minute = 8;
9  int second = 50;
10 /* USER CODE END PV */
11
12 /* USER CODE BEGIN PFP */
13 void display7SEG(int num);
14 void update7SEG(int index);
15 void updateClockBuffer(void);
16 /* USER CODE END PFP */
17
18 /* USER CODE BEGIN 0 */
19 int timer0_counter = 0;
20 int timer0_flag = 0;
21 int timer_7segcount = 4;
22 int TIMER_CYCLE = 10;
23 void setTimer0(int duration) {
24     timer0_counter = duration / TIMER_CYCLE;
25     timer0_flag = 0;
26 }
27 void timer_run(){
28     if (timer0_counter > 0) {
29         timer0_counter--;
30         if (timer0_counter == 0) timer0_flag = 1;
31     }
32 }
33 /* USER CODE END 0 */
34
35 /* USER CODE BEGIN 2 */
36 // Update display buffer once before starting timer interrupts
37 updateClockBuffer();
38
39 // Start Timer 2 in interrupt mode
40 HAL_TIM_Base_Start_IT(&htim2);
41 /* USER CODE END 2 */
42
43 /* USER CODE BEGIN WHILE */
44 setTimer0(250);
45 while (1)
46 {
47     if (timer0_flag == 1) {
48         update7SEG(index_led);

```

```

49
50     index_led++;
51
52     if (index_led >= MAX_LED) {
53         index_led = 0;
54     }
55
56     timer_7segcount--;
57
58     if (timer_7segcount <= 0) {
59         timer_7segcount = 4;
60         second++;
61         if (second >= 60)
62         {
63             second = 0;
64             minute++;
65         }
66
67         if (minute >= 60)
68         {
69             minute = 0;
70             hour++;
71         }
72
73         if (hour >= 24)
74         {
75             hour = 0;
76         }
77
78         updateClockBuffer();
79
80         HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
81     }
82
83     setTimer0(250);
84 }
85 /* USER CODE END WHILE */
86
87 /* USER CODE BEGIN 3 */
88 }
89
90 /* USER CODE BEGIN 4 */
91 void updateClockBuffer(void)
92 {
93     led_buffer[0] = hour / 10;
94     led_buffer[1] = hour % 10;
95     led_buffer[2] = minute / 10;
96     led_buffer[3] = minute % 10;
97 }

```

```

98
99 void display7SEG(int num) {
100     uint16_t seg_pins[7] = {
101         SEG0_Pin, SEG1_Pin, SEG2_Pin,
102         SEG3_Pin, SEG4_Pin, SEG5_Pin, SEG6_Pin
103     };
104
105     uint8_t seg_data[10][7] = {
106         {0,0,0,0,0,0,1}, // 0
107         {1,0,0,1,1,1,1}, // 1
108         {0,0,1,0,0,1,0}, // 2
109         {0,0,0,0,1,1,0}, // 3
110         {1,0,0,1,1,0,0}, // 4
111         {0,1,0,0,1,0,0}, // 5
112         {0,1,0,0,0,0,0}, // 6
113         {0,0,0,1,1,1,1}, // 7
114         {0,0,0,0,0,0,0}, // 8
115         {0,0,0,0,1,0,0}  // 9
116     };
117
118     if (num >= 0 && num <= 9) {
119         for (int i = 0; i < 7; i++) {
120             if (seg_data[num][i] == 0) {
121                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_RESET);
122             } else {
123                 HAL_GPIO_WritePin(GPIOB, seg_pins[i],
GPIO_PIN_SET);
124             }
125         }
126     }
127 }
128
129 void update7SEG(int index) {
130     switch (index) {
131         case 0:
132             display7SEG(led_buffer[0]);
133             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
134             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
135             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
136             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
137             break;
138         case 1:
139             display7SEG(led_buffer[1]);
140             HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
141             HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
142             HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
143             HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
144             break;

```

```

145     case 2:
146         display7SEG(led_buffer[2]);
147         HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
148         HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
149         HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
150         HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_SET);
151         break;
152     case 3:
153         display7SEG(led_buffer[3]);
154         HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
155         HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
156         HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_SET);
157         HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);
158         break;
159     default:
160         break;
161 }
162 }
163
164 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
165     if (htim->Instance == TIM2) {
166         timer_run();
167     }
168 }
169
170 /* USER CODE END 4 */

```

Program 1.13: Source code for Exercise 8

1.9 Exercise 9

This is an extra works for this lab. A LED Matrix is added to the system. A referenced design is shown in figure below:

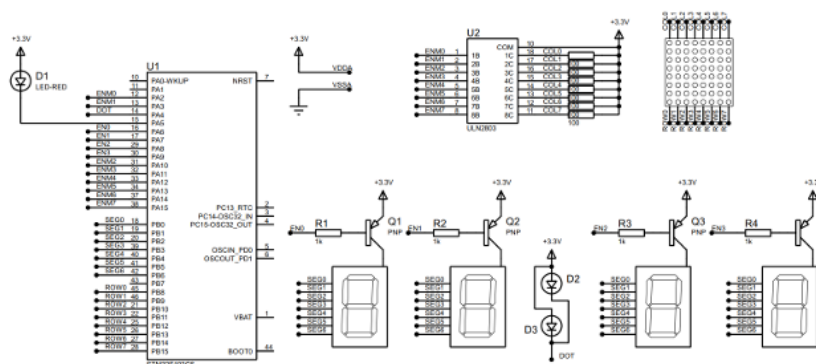


Figure 1.19: LED matrix is added to the simulation

In this schematic, two new components are added, including the **MATRIX-8X8-RED**

and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.

Report 2: Implement the function, `updateLEDMatrix(int index)`, which is similarly to 4 seven led segments.

STM32CubeIDE Code

```
1  const int MAX_LED_MATRIX = 8;
2  int index_led_matrix = 0;
3  uint8_t matrix_buffer[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06,
   0x07, 0x08};
4
5  void updateLEDMatrix(int index){
6      switch (index){
7          case 0:
8              break;
9          case 1:
10             break;
11          case 2:
12             break;
13          case 3:
14             break;
15          case 4:
16             break;
17          case 5:
18             break;
19          case 6:
20             break;
21          case 7:
22             break;
23          default:
24             break;
25      }
26 }
```

Program 1.14: Function to display data on LED Matrix

Students are free to choose the invoking frequency of this function. However, this function is supposed to be invoked in the main function. Finally, please update the **matrix_buffer** to display character "A".

Proteus Schematic

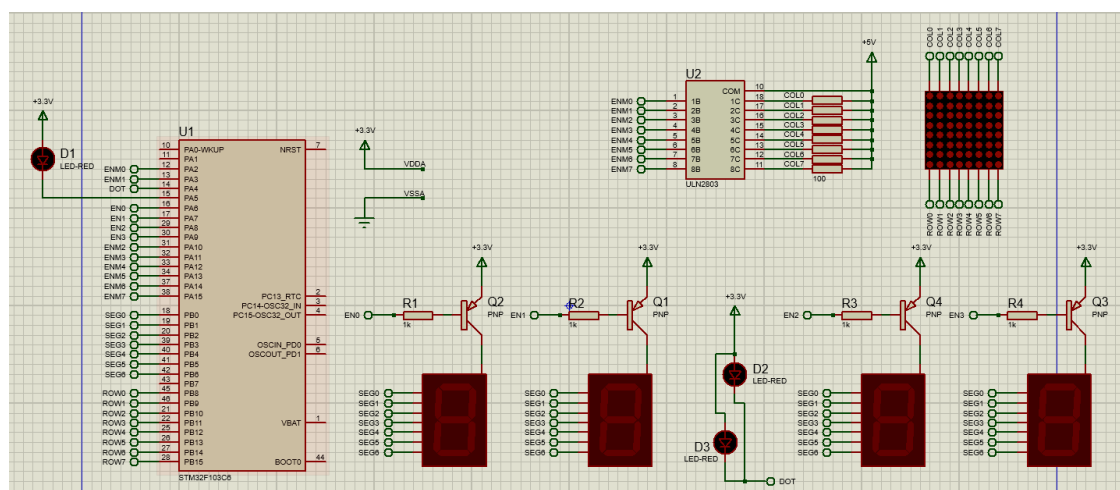


Figure 1.20: Schematic from Proteus for Exercise 9

STM32CubeIDE Configuration

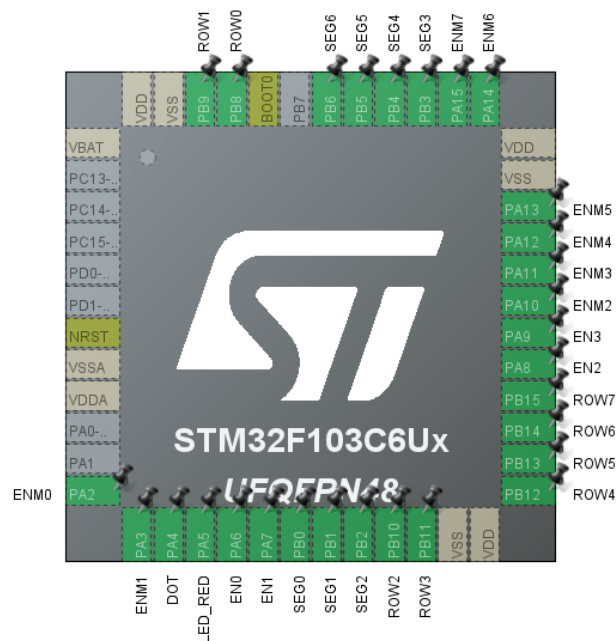


Figure 1.21: GPIO configuration for Exercise 9

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
2 const int MAX_LED_MATRIX = 8;
3 int index_led_matrix = 0;
4
```

```

5 uint8_t matrix_buffer[8] = {
6     0x00,
7     0xFC,
8     0xFE,
9     0x33,
10    0x33,
11    0xFE,
12    0xFC,
13    0x00
14 };
15 /* USER CODE END PV */
16
17 /* USER CODE BEGIN PFP */
18 void updateLEDMatrix(int index);
19 /* USER CODE END PFP */
20
21 /* USER CODE BEGIN WHILE */
22 while (1)
23 {
24     updateLEDMatrix(index_led_matrix);
25
26     index_led_matrix++;
27
28     if (index_led_matrix >= MAX_LED_MATRIX)
29     {
30         index_led_matrix = 0;
31     }
32
33     HAL_Delay(1);
34 /* USER CODE END WHILE */
35
36 /* USER CODE BEGIN 3 */
37 }
38
39 /* USER CODE BEGIN 4 */
40 void updateLEDMatrix(int index)
41 {
42     uint16_t row_pins[8] = {
43         ROW0_Pin, ROW1_Pin, ROW2_Pin, ROW3_Pin,
44         ROW4_Pin, ROW5_Pin, ROW6_Pin, ROW7_Pin
45     };
46
47     uint16_t enm_pins[8] = {
48         ENM0_Pin, ENM1_Pin, ENM2_Pin, ENM3_Pin,
49         ENM4_Pin, ENM5_Pin, ENM6_Pin, ENM7_Pin
50     };
51
52     if (index >= 0 && index < MAX_LED_MATRIX)
53     {

```

```

54     // Tat tat ca column
55     // ULN ON -> COL bi keo xuong LOW
56     HAL_GPIO_WritePin(GPIOA,
57                         ENM0_Pin | ENM1_Pin | ENM2_Pin |
ENM3_Pin |
58                         ENM4_Pin | ENM5_Pin | ENM6_Pin |
ENM7_Pin,
59                         GPIO_PIN_SET);
60
61     // ROW: LED sang khi ROW = LOW
62     for (int i = 0; i < 8; i++)
63     {
64         if (matrix_buffer[index] & (1 << i))
65             HAL_GPIO_WritePin(GPIOB, row_pins[i],
GPIO_PIN_RESET);
66         else
67             HAL_GPIO_WritePin(GPIOB, row_pins[i],
GPIO_PIN_SET);
68     }
69
70     // Chon column
71     // ENM LOW -> ULN OFF -> COL duoc keo len +5V
72     HAL_GPIO_WritePin(GPIOA, enm_pins[index],
GPIO_PIN_RESET);
73 }
74 }
75 /* USER CODE END 4 */

```

Program 1.15: Function to display data on LED Matrix

1.10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

Report 1: Briefly describe your solution and present your source code in the report.

The LED matrix animation is implemented by maintaining an `animation_buffer` containing the character pattern and an `animation_index` that determines the current horizontal shift. The `updateAnimationBuffer()` function updates `matrix_buffer` according to `animation_index`. The LED matrix is scanned continuously in the main loop using `updateLEDMatrix()`. After every 8-column scanning cycle, `animation_counter` is increased. When it reaches 10, `animation_index` is increased and the display buffer is updated, creating a left-shifting animation of the character.

Proteus Schematic

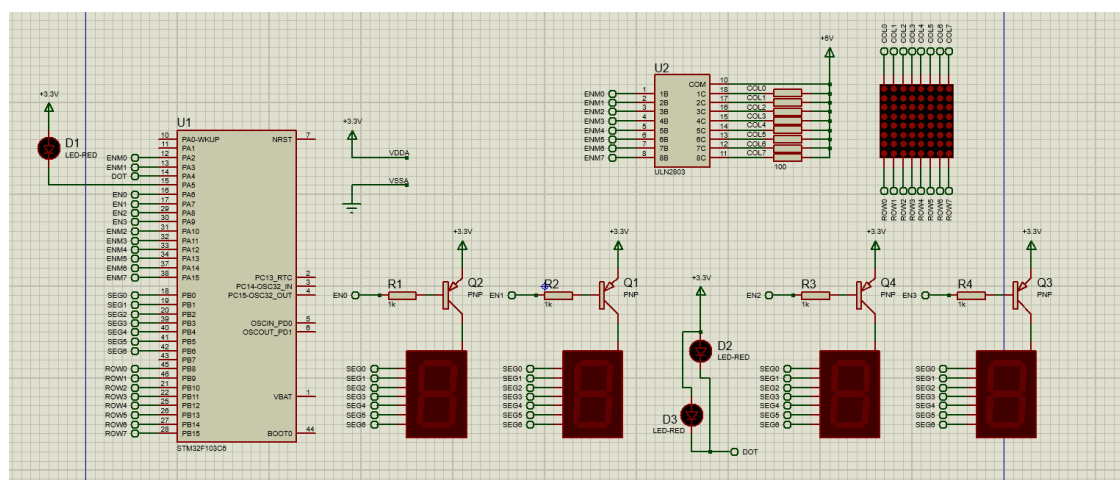


Figure 1.22: Schematic from Proteus for Exercise 10

STM32CubeIDE Configuration

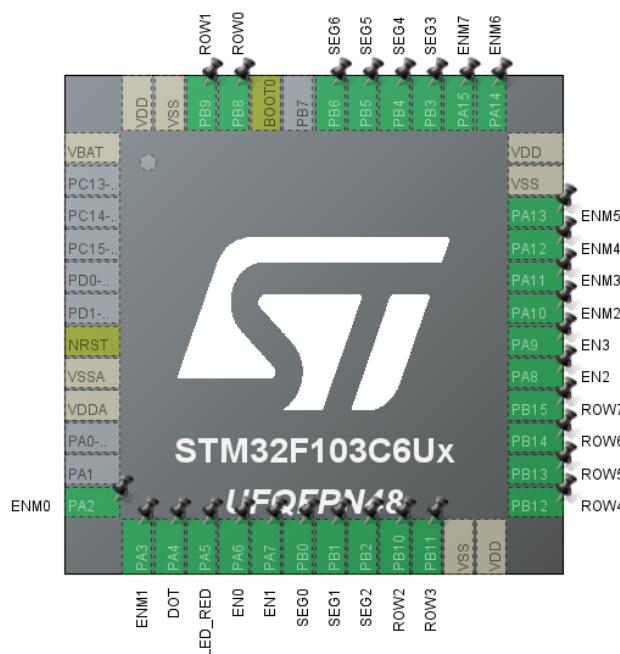


Figure 1.23: GPIO configuration for Exercise 10

STM32CubeIDE Code

```
1 /* USER CODE BEGIN PV */
2
3 const int MAX_LED_MATRIX = 8;
4 int index_led_matrix = 0;
```

```

5
6 uint8_t matrix_buffer[8] = {
7     0x00,
8     0xFC,
9     0xFE,
10    0x33,
11    0x33,
12    0xFE,
13    0xFC,
14    0x00
15 };
16
17 uint8_t animation_buffer[8] = {
18     0x00,
19     0xFC,
20     0xFE,
21     0x33,
22     0x33,
23     0xFE,
24     0xFC,
25     0x00
26 };
27
28 int animation_index = 0;
29 int animation_counter = 0;
30 /* USER CODE END PV */
31
32 /* USER CODE BEGIN PFP */
33 void updateLEDMatrix(int index);
34 /* USER CODE END PFP */
35
36 /* USER CODE BEGIN 0 */
37 void updateAnimationBuffer(void)
38 {
39     for (int i = 0; i < 8; i++)
40     {
41         int pos = animation_index + i;
42
43         if (pos >= 8)
44         {
45             pos -= 8;
46         }
47
48         matrix_buffer[i] = animation_buffer[pos];
49     }
50 }
51 /* USER CODE END 0 */
52
53 /* USER CODE BEGIN 2 */

```

```

54 updateAnimationBuffer();
55 /* USER CODE END 2 */
56
57 /* USER CODE BEGIN WHILE */
58 while (1)
59 {
60     updateLEDMatrix(index_led_matrix);
61
62     index_led_matrix++;
63
64     if (index_led_matrix >= MAX_LED_MATRIX)
65     {
66         index_led_matrix = 0;
67
68         animation_counter++;
69
70         if (animation_counter >= 10)
71         {
72             animation_counter = 0;
73
74             animation_index++;
75
76             if (animation_index >= 8)
77             {
78                 animation_index = 0;
79             }
80
81             updateAnimationBuffer();
82         }
83     }
84
85     HAL_Delay(1);
86 /* USER CODE END WHILE */
87
88 /* USER CODE BEGIN 3 */
89 }
90
91 /* USER CODE BEGIN 4 */
92 void updateLEDMatrix(int index)
93 {
94     uint16_t row_pins[8] = {
95         ROW0_Pin, ROW1_Pin, ROW2_Pin, ROW3_Pin,
96         ROW4_Pin, ROW5_Pin, ROW6_Pin, ROW7_Pin
97     };
98
99     uint16_t enm_pins[8] = {
100         ENM0_Pin, ENM1_Pin, ENM2_Pin, ENM3_Pin,
101         ENM4_Pin, ENM5_Pin, ENM6_Pin, ENM7_Pin
102     };

```

```

103
104     if (index >= 0 && index < MAX_LED_MATRIX)
105     {
106
107         HAL_GPIO_WritePin(GPIOA,
108                             ENM0_Pin | ENM1_Pin | ENM2_Pin |
109     ENM3_Pin |
110                             ENM4_Pin | ENM5_Pin | ENM6_Pin |
111     ENM7_Pin,
112                             GPIO_PIN_SET);
113
114         for (int i = 0; i < 8; i++)
115         {
116             if (matrix_buffer[index] & (1 << i))
117                 HAL_GPIO_WritePin(GPIOB, row_pins[i],
118                 GPIO_PIN_RESET);
119             else
120                 HAL_GPIO_WritePin(GPIOB, row_pins[i],
121                 GPIO_PIN_SET);
122         }
123
124         HAL_GPIO_WritePin(GPIOA, enm_pins[index],
125         GPIO_PIN_RESET);
126     }
127 }
128
129 /* USER CODE END 4 */

```

Program 1.16: Source code for Exercise 10